

Giáo Án

Phiên bản web: <https://chief-heliotrope-d20.notion.site/Gi-o-n-1dcb9e967cbc807c883de8f68133c233?pvs=4>

 Lưu ý cho giảng viên:

- **Sử dụng mã nguồn Bài 9:** Giáo án này xây dựng dựa trên ứng dụng Todo App hoàn chỉnh với Redux Toolkit, Thunk, và Authentication từ Bài 9. Đảm bảo học viên có sẵn mã nguồn này để thực hành.
- **Công cụ:** Khuyến khích sử dụng **React DevTools Profiler** để trực quan hóa và đo lường hiệu năng trước và sau khi áp dụng các kỹ thuật tối ưu. Nhấn mạnh rằng việc tối ưu nên dựa trên đo lường thực tế, không phải cảm tính.
- **Tránh tối ưu quá sớm:** Nhắc nhở học viên rằng việc tối ưu hóa đi kèm với sự phức tạp trong code. Chỉ nên tối ưu khi thực sự cần thiết (ví dụ: ứng dụng chậm, có bottleneck rõ ràng).

Tại Sao Cần Tối Ưu và Các Vấn Đề Thường Gặp

1. Mục tiêu

- Học viên hiểu được tầm quan trọng của việc tối ưu hóa hiệu năng ứng dụng React (trải nghiệm người dùng, tốc độ tải).
- Học viên nhận diện được các vấn đề hiệu năng phổ biến: kích thước bundle lớn ban đầu và re-render không cần thiết.
- Học viên biết được các công cụ cơ bản để đo lường hiệu năng (React DevTools Profiler, Network Tab).

2. Nội dung

2.1 Tại sao phải tối ưu?

- **Trải nghiệm người dùng (UX):** Ứng dụng nhanh, mượt mà mang lại trải nghiệm tốt hơn.
- **Tỷ lệ chuyển đổi:** Tốc độ tải trang ảnh hưởng trực tiếp đến việc giữ chân người dùng.
- **SEO:** Google ưu tiên các trang web tải nhanh.

2.2 Các vấn đề hiệu năng phổ biến trong React

- **Kích thước Bundle lớn:** Toàn bộ mã nguồn JavaScript của ứng dụng được tải cùng lúc khi người dùng truy cập lần đầu, gây chậm trễ (đặc biệt trên mạng yếu).
- **Re-render không cần thiết:** Component bị render lại ngay cả khi dữ liệu hoặc giao diện của nó không hề thay đổi, gây lãng phí tài nguyên CPU. Điều này thường xảy ra khi component cha re-render hoặc nhận các props (đặc biệt là functions hoặc objects) có tham chiếu thay đổi.

2.3 Công cụ đo lường

- **Browser DevTools - Network Tab:** Kiểm tra kích thước các file JavaScript được tải.
- **React DevTools - Profiler Tab:** Ghi lại và phân tích các lần render, xác định component nào re-render thường xuyên hoặc tốn thời gian.
 - **(Demo nhanh cách dùng Profiler)** Ghi lại một vài tương tác trong Todo App (thêm, toggle todo) và xem kết quả.
-  Lưu ý cho giảng viên
 - Nhấn mạnh việc **"Measure First"** (Đo lường trước). Không tối ưu mù quáng.

Giảm Kích Thước Bundle với Code Splitting (**React.lazy** và **Suspense**)

1. Mục tiêu

- Học viên hiểu khái niệm Code Splitting và lợi ích của nó trong việc giảm thời gian tải ban đầu.
- Học viên biết cách sử dụng `React.lazy` để tải component theo yêu cầu (route-based code splitting).
- Học viên biết cách sử dụng `Suspense` để hiển thị fallback UI trong khi component đang được tải.
- Học viên áp dụng được kỹ thuật này vào ứng dụng Todo App (cho các trang `Login` , `Register` , `Home`).

2. Nội dung

2.1 Code Splitting là gì?

- Thay vì gộp toàn bộ code vào một file bundle duy nhất, Code Splitting chia bundle thành các phần nhỏ hơn (chunks).
- Các chunk này có thể được tải **theo yêu cầu** (on-demand) hoặc **song song**, giúp giảm kích thước tải ban đầu và cải thiện Time-to-Interactive (TTI).

2.2 `React.lazy`

- Một hàm của React cho phép bạn render một component được import động (dynamic import) như một component thông thường.
- Cú pháp: `const MyComponent = React.lazy(() => import('./MyComponent'));`
- `import()` trả về một Promise, resolve thành một module có export default chứa component React.

2.3 `Suspense`

- Component cho phép bạn chỉ định một giao diện "chờ" (fallback UI, ví dụ: spinner, skeleton) trong khi các component con bên trong nó đang được tải bằng `React.lazy` .
- Cú pháp:

```
import React, { Suspense } from 'react';
const OtherComponent = React.lazy(() => import('./OtherComponent'));

function MyComponent() {
  return (
    <div>
      <Suspense fallback={<div>Loading...</div>}>
        <OtherComponent />
      </Suspense>
    </div>
  );
}
```

2.4 Áp dụng vào Todo App (Route-based Code Splitting)

- Mở file `src/App.jsx` .
- Sử dụng `React.lazy` để import các component màn hình (`Home` , `Login` , `Register`).
- Bọc `Routes` hoặc từng `Route` bằng `Suspense` và cung cấp `fallback` UI.

```
// src/App.jsx (Ví dụ cập nhật)
import React, { Suspense } from 'react'; // Thêm Suspense
import { BrowserRouter, Routes, Route } from 'react-router-dom';

// Import components bằng React.lazy
const Home = React.lazy(() => import('./screens/Home/Home'));
const Login = React.lazy(() => import('./screens/Login/Login'));
const Register = React.lazy(() => import('./screens/Register/Register'));
const ProtectedRoute = React.lazy(() => import('./components/ProtectedRoute/ProtectedRoute')); // Cũng có thể lazy load

function App() {
```

```

return (
  <BrowserRouter>
    {/* Bọc toàn bộ Routes bằng Suspense */}
    <Suspense fallback={<div>Loading Page...</div>}>
      <Routes>
        {/* Các route công khai */}
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />

        {/* Route được bảo vệ */}
        <Route
          path="/"
          element={
            <ProtectedRoute> {/* ProtectedRoute cũng có thể được lazy load */}
              <Home />
            </ProtectedRoute>
          }
        />
      </Routes>
    </Suspense>
  </BrowserRouter>
);
}

export default App;

```

- **Kiểm tra kết quả:** Mở Network Tab trong DevTools, xóa cache và tải lại trang. Quan sát các file JavaScript nhỏ hơn được tải khi điều hướng giữa các trang thay vì một file lớn ban đầu.

📌 Lưu ý cho giảng viên:

- Giải thích rõ `React.lazy` hiện chỉ hỗ trợ default exports. Nếu component được export dạng named export, cần tạo file trung gian để re-export default.
- `Suspense` có thể đặt ở nhiều cấp độ khác nhau trong cây component.
- Nhấn mạnh đây là cách tối ưu hiệu quả cho thời gian tải *ban đầu*.

Ngăn Chặn Re-render Không Cần Thiết với `React.memo`

1. Mục tiêu

- Học viên hiểu được cơ chế re-render mặc định của React và tại sao nó có thể dẫn đến re-render không cần thiết.
- Học viên biết cách sử dụng `React.memo` để tối ưu hóa functional component, ngăn chặn re-render nếu props không thay đổi.
- Học viên xác định được các component ứng viên tốt cho `React.memo` trong Todo App (ví dụ: `TodoItem`).
- Học viên áp dụng `React.memo` và quan sát sự thay đổi (qua `console.log` hoặc Profiler).

2. Nội dung

2.1 Tại sao component re-render?

- State của component thay đổi (qua `useState` hoặc `useReducer`).
- Props của component thay đổi.
- Component cha re-render (đây là lý do phổ biến gây re-render không cần thiết ở component con).

2.2 `React.memo` là gì?

- Là một Higher-Order Component (HOC). Nó bọc component của bạn và "ghi nhớ" (memoizes) kết quả render.

- React sẽ bỏ qua việc render lại component nếu props của nó **không thay đổi** (so sánh nông - shallow comparison).
- Chỉ dùng cho **functional components**. (Class components có `PureComponent` hoặc `shouldComponentUpdate`).
- **Cú pháp:**

```
const MyComponent = React.memo(function MyComponent(props) {
  /* render using props */
});
// Hoặc
function MyComponent(props) { /* ... */ }
export default React.memo(MyComponent);
```

2.3 Shallow Comparison (So sánh nông)

- `React.memo` chỉ so sánh các giá trị props ở cấp độ đầu tiên.
- Với kiểu dữ liệu nguyên thủy (string, number, boolean): So sánh giá trị.
- Với kiểu dữ liệu phức tạp (object, array, function): So sánh **tham chiếu (reference)**. Nếu tham chiếu thay đổi (dù nội dung bên trong giống hệt), `React.memo` sẽ coi là props đã thay đổi.

2.4 Áp dụng vào Todo App

- **Tối ưu `TodoItem`** : Nó nhận prop `todo` (object). Nếu `TodoList` re-render vì lý do khác (ví dụ: state loading thay đổi) nhưng một `TodoItem` cụ thể vẫn nhận cùng một object `todo` (tham chiếu không đổi), nó không cần re-render.
- Bọc `TodoItem` bằng `React.memo` .

```
// src/components/TodoItem/TodoItem.jsx
import React from 'react';
import { useDispatch } from 'react-redux';
// import { toggleTodo, deleteTodo } from '../redux/slices/todoSlice'; // Sửa đường dẫn import nếu cần
import { toggleTodo, deleteTodo } from '../redux/slices/todoSlice'; // Đường dẫn tương đối từ TodoItem

// Bọc component bằng React.memo
const TodoItem = React.memo(function TodoItem({ todo }) { // Đặt tên cho function bên trong memo để dễ debug
  console.log("Rendering TodoItem:", todo.id); // Thêm log để quan sát
  const dispatch = useDispatch();

  const handleToggleTodo = () => {
    dispatch(toggleTodo(todo.id));
  };

  const handleDeleteTodo = () => {
    dispatch(deleteTodo(todo.id));
  };

  return (
    <li>
      <input
        type="checkbox"
        checked={todo.completed}
        onChange={handleToggleTodo}
      />
      <span style={{ textDecoration: todo.completed ? 'line-through' : 'none' }}>
        {todo.text}
      </span>
      <button onClick={handleDeleteTodo}>Delete</button>
    </li>
  );
});
```

```
}); // Đóng React.memo
```

```
export default TodoItem;
```

- **Quan sát:**
 - Mở console, thực hiện các hành động (thêm, xóa, toggle).
 - Quan sát log "Rendering TodoItem: ...".
 - Lý tưởng nhất là khi toggle 1 todo, chỉ todo đó và `TodoList` re-render, các `TodoItem` khác không re-render (nếu tham chiếu `todo` của chúng không đổi). Sử dụng Profiler để thấy rõ hơn.
- 📌 **Lưu ý cho giảng viên**
 - Giải thích kỹ về **shallow comparison** và hạn chế của nó với object/array/function props. Đây là tiền đề cho phần tiếp theo (`useCallback` , `useMemo`).
 - `React.memo` có thể nhận một hàm so sánh tùy chỉnh làm đối số thứ hai, nhưng nên hạn chế sử dụng vì có thể làm code phức tạp hơn.
 - Nhấn mạnh rằng `React.memo` không phải là "viên đạn bạc", nó có chi phí (so sánh props). Chỉ dùng khi component render tốn kém hoặc re-render thường xuyên không cần thiết.

Ổn Định Tham Chiếu với `useCallback` và `useMemo`

1. Mục tiêu

- Học viên hiểu rõ vấn đề **referential equality** (bình đẳng tham chiếu) và tại sao nó làm `React.memo` không hiệu quả với props là functions hoặc objects/arrays.
- Học viên biết cách sử dụng `useCallback` để memoize (ghi nhớ) các hàm callback, đảm bảo chúng giữ nguyên tham chiếu giữa các lần render (nếu dependencies không đổi).
- Học viên biết cách sử dụng `useMemo` để memoize các giá trị tính toán phức tạp hoặc các object/array được tạo ra trong quá trình render.
- Học viên hiểu cách kết hợp `useCallback` , `useMemo` với `React.memo` để tối ưu hóa hiệu quả.

2. Nội dung

2.1 Vấn đề Referential Equality

- Trong JavaScript, `{ } === { }` là `false` , `[] === []` là `false` , `() => { } === () => { }` là `false` .
- Khi một component cha định nghĩa một hàm hoặc tạo một object/array bên trong thân hàm và truyền nó làm prop cho component con (đã được bọc `React.memo`), component con sẽ nhận một **tham chiếu mới** của prop đó mỗi khi cha re-render.
- Điều này làm cho việc so sánh nông của `React.memo` luôn trả về `false` , dẫn đến component con bị re-render không cần thiết mặc dù logic hoặc dữ liệu không thay đổi.

2.2 `useCallback`

- Hook nhận vào một hàm inline và một mảng dependencies.
- Trả về một **phiên bản memoized** của hàm đó. Phiên bản này chỉ thay đổi nếu một trong các dependencies thay đổi.
- Hữu ích khi truyền callbacks xuống các component con được tối ưu hóa (`React.memo`).
- Cú pháp: `const memoizedCallback = useCallback(fn, [deps]);`

```
// Ví dụ: Nếu TodoList truyền hàm handleToggle xuống TodoItem
function TodoList() {
  const dispatch = useDispatch();
  const todos = useSelector(state => state.todos.todos);

  // Nếu truyền hàm này xuống TodoItem
```

```

const handleToggle = useCallback((id) => {
  dispatch(toggleTodo(id));
}, [dispatch]); // dispatch từ react-redux là stable

// Nếu truyền hàm này xuống TodoItem
const handleDelete = useCallback((id) => {
  dispatch(deleteTodo(id));
}, [dispatch]);

return (
  <ul>
    /* Giả sử TodoItem nhận onToggle và onDelete props */
    /* {todos.map(todo => (
      <TodoItem key={todo.id} todo={todo} onToggle={handleToggle} onDelete={handleDelete} />
    ))} */
    /* Trong code gốc, TodoItem tự dùng dispatch nên không cần truyền callback */
  </ul>
);
}

```

2.3 useMemo

- Hook nhận vào một hàm "create" và một mảng dependencies.
- Trả về một **giá trị memoized** là kết quả của việc gọi hàm "create".
- `useMemo` sẽ chỉ tính toán lại giá trị memoized khi một trong các dependencies thay đổi.
- Hữu ích để tránh tính toán lại các giá trị tốn kém trên mỗi lần render, hoặc để memoize object/array truyền xuống component con.
- Cú pháp: `const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);`

```

// Ví dụ: Tính toán số lượng todo chưa hoàn thành trong TodoList (ví dụ đơn giản)
function TodoList() {
  const todos = useSelector(state => state.todos.todos);
  // ... các state khác

  const incompleteTodosCount = useMemo(() => {
    console.log("Calculating incomplete count..."); // Log để xem khi nào tính toán lại
    return todos.filter(todo => !todo.completed).length;
  }, [todos]); // Chỉ tính lại khi mảng todos thay đổi

  return (
    <div>
      <p>Incomplete Todos: {incompleteTodosCount}</p>
      <ul>
        /* ... render list */
      </ul>
    </div>
  );
}

```

- **Kết hợp với `React.memo`**: Khi component cha sử dụng `useCallback` cho các hàm và `useMemo` cho các object/array truyền làm props xuống component con đã bọc `React.memo`, việc tối ưu re-render sẽ hiệu quả hơn vì các props này sẽ giữ nguyên tham chiếu nếu dependencies của chúng không đổi.
- 📌 **Lưu ý cho giảng viên:**
 - **Quan trọng:** Giải thích rõ về **mảng dependencies (`deps`)**. Nếu bỏ sót dependency, hàm/giá trị sẽ không được cập nhật đúng → bug. Nếu thêm dependency không cần thiết, sẽ mất tác dụng memoization.
 - `useCallback(fn, deps)` tương đương với `useMemo(() => fn, deps)`.

- `useCallback` và `useMemo` cũng có chi phí (bộ nhớ, so sánh dependencies). **Chỉ sử dụng khi có lợi ích rõ ràng** (ví dụ: tối ưu component con với `React.memo`, tránh tính toán nặng). Không nên lạm dụng.
- Giảng viên có thể sửa đổi code `TodoItem` để nhận `onToggle`, `onDelete` props từ `TodoList` để minh họa rõ hơn sự cần thiết của `useCallback`.

Tổng kết và Các Kỹ Thuật Khác

1. Mục tiêu

- Học viên ôn lại các kỹ thuật tối ưu đã học (Lazy Loading, Memo, Callback, Memo).
- Học viên nắm được nguyên tắc chung khi tối ưu: Đo lường trước, tránh tối ưu sớm.
- (Tùy chọn) Học viên biết thêm về các kỹ thuật khác như Virtualization (virtualized lists).

2. Nội dung

2.1 Tóm tắt các kỹ thuật

- `React.lazy` + `Suspense`: Giảm bundle size ban đầu, cải thiện tốc độ tải trang (cho routes hoặc component lớn).
- `React.memo`: Ngăn functional component re-render nếu props không đổi (so sánh nông).
- `useCallback`: Memoize hàm callback, ổn định tham chiếu cho props là function.
- `useMemo`: Memoize giá trị tính toán hoặc object/array, ổn định tham chiếu và tránh tính toán lại.

2.2 Nguyên tắc vàng

- **Đo lường trước khi tối ưu:** Dùng Profiler để xác định bottleneck.
- **Tránh tối ưu quá sớm (Premature Optimization):** Đừng làm phức tạp code khi chưa cần thiết.
- **Hiểu rõ trade-offs:** Tối ưu thường đánh đổi sự đơn giản lấy hiệu năng.

2.3 Virtualization

- Đối với danh sách rất dài (hàng trăm, hàng nghìn items), việc render tất cả các DOM node cùng lúc sẽ rất chậm.
- **Virtualized lists** (sử dụng thư viện như `react-window` hoặc `react-virtualized`) chỉ render những items đang hiển thị trong viewport (và một vài item đệm xung quanh), giúp cải thiện đáng kể hiệu năng render danh sách lớn.
- Giới thiệu sơ qua khái niệm và khi nào nên dùng.
- 📌 **Lưu ý cho giảng viên:**
 - Khuyến khích học viên thực hành đo lường bằng Profiler trên chính ứng dụng Todo App sau khi áp dụng các kỹ thuật.
 - Chia sẻ kinh nghiệm thực tế về việc khi nào nên và không nên tối ưu.
 - 🖱️ Cho học viên làm **Bài Tập Cuối Chương**.